

CASE STUDY 1

Smart Payment Processing System

Scenario

An e-commerce platform supports multiple payment gateways.

At runtime, the system must choose the appropriate payment implementation.

The application should:

- Process payment
- Log transaction
- Initialize payment gateway connection
- Close resources on shutdown

Student Requirements

1 Create Interface

PaymentService

+ processPayment(double amount)

2 Create Implementations

Class	Requirement
CreditCardPayment	Default payment method

UPIPayment Secondary payment
method

Annotation Rules

- Both → @Component
 - CreditCardPayment → @Primary
 - UPIPayment → regular @Component
-

3 Create Logger Component

TransactionLogger

- Annotate with @Component
 - Inject into PaymentService implementation using **Constructor Injection**
 - Add:
 - @PostConstruct → Print: "Logger initialized"
 - @PreDestroy → Print: "Logger destroyed"
-

4 Create PaymentProcessor

PaymentProcessor

Responsibilities:

- Depends on PaymentService

- Depends on `TransactionLogger`

Injection Rules

- Inject `PaymentService` using **Constructor Injection**
 - Inject `TransactionLogger` using **Field Injection with `@Autowired`**
 - Use `@Qualifier` to explicitly inject `UpiPayment` instead of primary
-

5 Scope & Lazy

- Make `UpiPayment` → `@Scope("prototype")`
- Make `CreditCardPayment` → `@Lazy`

Students must observe:

- Prototype bean lifecycle behavior
 - Lazy initialization behavior
-

6 Configuration Class

Create:

AppConfig

- Annotate with `@Configuration`
 - Add `@ComponentScan("your.base.package")`
-

Concepts Covered

- `@Component`
- `@Primary`
- `@Qualifier`
- `@Scope`
- `@Lazy`
- `@Autowired`
- Constructor DI
- Field DI
- `@PostConstruct`
- `@PreDestroy`
- `@Configuration`
- `@ComponentScan`

✓ CASE STUDY 2

Online Food Delivery System

Scenario

A food delivery application manages:

- Restaurant Service
- Delivery Service
- Notification Service (Email & SMS)
- Order Service

Only one notification service should be default.

Student Requirements

1 Create NotificationService Interface

sendNotification(String message)

2 Implementations

Class	Requirement
EmailNotification	Default
SmsNotification	Secondary

Rules

- Both → `@Component`
 - EmailNotification → `@Primary`
 - SmsNotification → regular component
-

3 DeliveryService

- Annotate with `@Component`
 - Use `@Scope("singleton")`
 - Add:
 - `@PostConstruct` → Print "Delivery Service Ready"
 - `@PreDestroy` → Print "Delivery Service Closed"
-

4 RestaurantService

- Annotate with `@Component`
 - Inject DeliveryService using **Setter Injection**
-

5 OrderService

- Annotate with `@Component`

- Inject:
 - NotificationService → Constructor Injection
 - RestaurantService → Field Injection
 - Use `@Qualifier("smsNotification")` to override primary
-

6 Make SmsNotification Lazy

Use:

`@Lazy`

Students should test:

- Bean creation timing
 - Default vs Qualifier resolution
-

7 Configuration

Create:

FoodAppConfig

- `@Configuration`
 - `@ComponentScan`
-

Concepts Covered

- Multiple implementations
 - `@Primary`
 - `@Qualifier`
 - Constructor + Setter + Field DI
 - `@Scope`
 - `@Lazy`
 - Lifecycle annotations
-

CASE STUDY 3

Banking Loan Approval System

Scenario

A banking system evaluates loan applications using different validation strategies.

System includes:

- `LoanValidator`
- `CreditScoreValidator`
- `IncomeValidator`
- `LoanService`
- `AuditService`

Student Requirements

1 Create Interface

LoanValidator

+ validateLoan(double amount)

2 Implement Implementations

Class	Requirement
CreditScoreValidator	Default validator
IncomeValidator	Secondary

Rules

- Both → @Component
 - CreditScoreValidator → @Primary
 - IncomeValidator → normal component
 - IncomeValidator → @Scope("prototype")
-

3 Create AuditService

- Annotate with @Component
- Use @Lazy

- Add:
 - `@PostConstruct`
 - `@PreDestroy`
-

4 Create LoanService

- Annotate with `@Component`
 - Inject:
 - LoanValidator using **Constructor Injection**
 - AuditService using **Setter Injection**
 - Use `@Qualifier("incomeValidator")` in constructor
-

5 Configuration Class

BankAppConfig

- `@Configuration`
- `@ComponentScan`